02 MAR 2026

AADK TASK 2

Nullability, Safe Calls & Robust Data Handling

MindMatrix

Research Key Concepts

Nullable Types (String?)

A nullable type in Kotlin is a type that can hold either a value or null. We add ? after the type name to make it nullable.

Example:
val name: String? = null

Here, name can either store a String value or be null.

Non-Nullable Types (String)

A non-nullable type cannot hold null. Kotlin forces us to initialize it properly, which prevents crashes.

Example:
val name: String = "Sanjana"

If we try to assign null here, the compiler will give an error.

Safe Call Operator (?.)

The safe call operator is used to safely access a property or function on a nullable object. If the object is null, it simply returns null instead of crashing.

Example:
val name: String? = "Kotlin"
val length = name?.length ?: 0

If name is null, length becomes 0.

Elvis Operator (?:)

The Elvis operator provides a default value if the left side is null.

Example:

```
val nickname: String? = null
val displayName = nickname ?: "Guest"
```

If nickname is null, it assigns "Guest".

Non-Null Assertion (!!)

AThe !! operator forces Kotlin to treat a nullable variable as non-null. If the value is actually null, it will crash with a NullPointerException (NPE).

Example:

```
val name: String? = "Android"
println(name!!.length)
```

This is risky and should be avoided unless we are 100% sure the value is not null.

Converting Nullable to Non-Nullable using if

```
val name: String? = getNameFromServer()

val finalName: String = if (name != null) {
    name
} else {
    "Unknown"
}
```

This ensures finalName is non-null.

Data Handling Scenario

Scenario: User Profile Fetch from Remote API

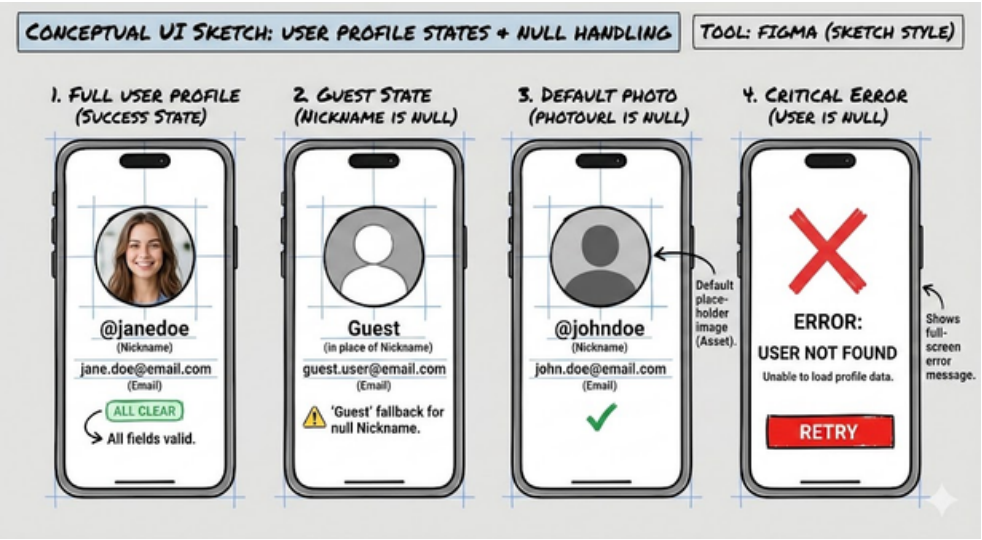
In this scenario, the app fetches a User? object from a remote server. Since network responses may have missing fields, some values like photoUrl, nickname, or even the entire user object might be null. Nullability is very important here because if we do not handle null properly, the app may crash.

Null-Safe Logic Document

Data Handling Table

Field Name	Type	Nullable?	Safe Handling Strategy	UI Fallback
user	User?	Yes	Check if null before accessing	Show error screen
nickname	String?	Yes	Use Elvis operator	Show "Guest"
photoUrl	String?	Yes	Check null before loading image	Show placeholder image
email	String?	Yes	Safe call + default text	Show "Not Available"

Compose UI Sketch & Modifier Plan



Handling Null States

- If user is null → Show full-screen error message.
- If nickname is null → Show "Guest".
- If photoUrl is null → Show default placeholder image.

Compose Code Snippet

```
@Composable
fun UserProfileScreen(user: User?) {
    if (user == null) {
        Text(text = "User data not available")
        return
    }
    Column {
        val displayName = user.nickname ?: "Guest"
        val displayEmail = user.email ?: "Not Available"
        Text(text = displayName)
        Text(text = displayEmail)
         if (user.photoUrl != null) {
          } else {
          }
    }
}
```

Reflection

Null safety is very important in Android apps because many crashes happen due to NullPointerExceptions. When we fetch data from network or database, we cannot guarantee that everything will be present. If we don't handle null properly, the app may crash suddenly, which gives a bad user experience. Kotlin's null safety system forces developers to think carefully about possible null values. Using safe calls and the Elvis operator improves app robustness and makes development more confident and productive.